

Documentación Técnica de Arquitectura

Infraestructura Web y Portfolio IT

Especificaciones Técnicas, Componentes y Despliegue CI/CD

Análisis integral del repositorio con implementaciones en código fuente

Autor:	Redwan Alexandru Teodor
Entorno Web:	<code>redwan-alexandru-teodor.github.io/inicio</code>
Repositorio:	<code>github.com/Redwan-Alexandru-Teodor/inicio</code>
Stack Base:	Next.js 16 + TypeScript + Tailwind CSS v4
Despliegue:	GitHub Pages vía GitHub Actions (Pipelines CI/CD)
Revisión:	Octubre 2026 (Versión 2.0.0)

Documentación exhaustiva de la arquitectura del repositorio: 10 componentes modulares React, 4 bibliotecas internas de utilidad, sistema de renderizado de datos, pipelines de integración continua (CI/CD), variables de entorno, topología de estilos globales, optimización SEO, auditoría de accesibilidad y guía de mantenimiento estructural.

Índice

1. Visión General de la Arquitectura	3
1.1. Propósito de la infraestructura	3
1.2. Stack tecnológico implementado	4
2. Topología de Archivos y Directorios	4
2.1. Árbol de directorios documentado	4
3. Configuración del Núcleo (lib/)	6
3.1. lib/site-config.ts — Variables globales	6
3.2. lib/utils.ts — Fusión de clases condicionales	6
3.3. lib/paises.ts — Matriz de validación internacional	6
3.4. lib/contact.ts — Controlador de peticiones del formulario	7
4. Análisis de Componentes de Interfaz	8
4.1. app/page.tsx — Ensamblado del layout principal	8
4.2. app/layout.tsx — Raíz del DOM y control de SEO	9
4.3. components/services.tsx — Matriz renderizada de servicios	9
4.4. components/proyectos.tsx — Subsistema de renderizado de proyectos .	10
5. Sistema de Procesamiento de Formularios	11
5.1. Validación asíncrona de estado por campo	11
5.2. Control de lectura obligatoria (bloqueo de envíos)	11
6. Implementación de Ofuscación Anti-Spam	11
6.1. Generación segura de identificadores y descriptores	12
7. Sistema de Configuración de Datos (Proyectos)	12
7.1. Interfaz y entidades (data/proyectos/types.ts)	12
8. Sistema de Renderizado y Estilos Globales	13
8.1. Asignación de variables nativas de CSS	13
9. Pipeline de Integración y Despliegue (CI/CD)	13
9.1. Análisis del motor YAML	13
10. Rendimiento, Optimización SEO y WCAG	15
10.1. Optimización de métricas centrales de rendimiento	15
10.2. Auditoría y directivas de accesibilidad	15

Índice de cuadros

1.	Matriz de dependencias base (<code>package.json</code>)	4
2.	Estructura de la interfaz de validación telefónica	7
3.	Funciones integradas en <code>proyectos.tsx</code>	10
4.	Protocolos de optimización de infraestructura	15

Índice de listados

1.	Estructura del repositorio en entorno de desarrollo	4
2.	<code>site-config.ts</code> — Enrutamiento y control de entornos	6
3.	<code>utils.ts</code> — Motor de resolución CSS	6
4.	Implementación del Server Action en <code>contact.ts</code>	7
5.	<code>page.tsx</code> — Orden jerárquico de renderizado	8
6.	Estructura de la iteración de servicios	9
7.	Control de interfaz de usuario en modales	10
8.	Asignación condicional de clases en entradas del formulario	11
9.	Lógica de comprobación de lectura de políticas de uso	11
10.	Procedimiento de ofuscación de credenciales	12
11.	Tipado central del motor de proyectos	12
12.	Declaración de tokens globales en <code>globals.css</code>	13
13.	Directivas del archivo <code>deploy.yml</code>	13

1. Visión General de la Arquitectura

1.1. Propósito de la infraestructura

La plataforma opera como el eje central del portfolio profesional y la pasarela de contacto unificada de Redwan Alexandru Teodor, especialista técnico en sistemas informáticos y redes. Los requerimientos funcionales del sistema son:

- Renderización estática de 8 servicios IT optimizados para despliegue sin servidor.
- Despliegue de un visor de proyectos con interpretación Markdown embebida y control de estados de interfaz.
- Gestión de ingesta de datos a través de un formulario con validación asíncrona avanzada.
- Implementación de protocolos de seguridad anti-spam mediante codificación en tiempo de ejecución (Base64) e identificadores únicos de sesión (UUID).
- Cumplimiento estricto de las normativas vigentes (RGPD y LSSI-CE) mediante validaciones bloqueantes en el flujo de usuario.

1.2. Stack tecnológico implementado

Cuadro 1. Matriz de dependencias base (package.json)

Categoría	Tecnología	Versión y especificaciones
Core Framework	Next.js	16.3.3 — SSG (Static Site Generation) estricto
Tipado	TypeScript	5.7.3 — Cobertura de tipado estático al 100 %
Estilización	Tailwind CSS	v4.3.3 — Variables CSS nativas y clases utilitarias
Animaciones	tw-animate-css	v1.4.0 — Transiciones declarativas y fade-ins
Primitivos UI	shadcn/ui	v4.11.0 — Componentes accesibles (Radix UI)
Engine de UI	@base-ui/react	v1.5.0 — Abstracción de botones
Variantes CSS	class-variance-authority	v0.7.1 — Función <code>cva()</code> para estados de UI
Fusión de clases	clsx + tailwind-merge	Resolución de conflictos de especificidad CSS
Iconografía	Lucide React	v1.16.0 — Renderizado vectorial SVG
Backend form	Formspree	Endpoint SaaS para orquestación de emails
Package manager	pnpm	v12.3.4 — Gestión eficiente de caché y dependencias
CI/CD pipeline	GitHub Actions	Automatización de despliegue sobre rama <code>main</code>

2. Topología de Archivos y Directorios

2.1. Árbol de directorios documentado

```
inicio-main/
|-- .github/workflows/deploy.yml    # Pipeline de despliegue e integración
    (CI/CD)
|-- app/
```

```

| |-- globals.css                # Variables globales nativas y espacio de
| color OKLCH
| |-- layout.tsx                 # Root DOM: fuentes, inyección de metadatos
| y JSON-LD
| |-- not-found.tsx             # Controlador de rutas no resueltas (Error
404)
| '-- page.tsx                  # Enrutador principal "/" e inyector de
dependencias
|-- components/
| |-- hero.tsx                  # Módulo principal (Hero) e inicialización
de CTA
| |-- about.tsx                 # Renderizado de metadatos personales
| |-- services.tsx              # Matriz interactiva de servicios
| |-- proyectos.tsx             # Motor de renderizado de galería y parseo
Markdown
| |-- contact.tsx               # Contenedor de la vista de contacto
| |-- contact-form.tsx          # Formulario con validación de estado en
tiempo real
| |-- mail-modal.tsx            # Componente anti-spam con descifrado en
cliente
| |-- site-header.tsx           # Navegación global con control de estado
(Sticky/Mobile)
| |-- site-footer.tsx           # Pie de página y abstracción de avisos
legales
| '-- ui/button.tsx             # Componente primitivo parametrizado
|-- data/proyectos/
| |-- types.ts                  # Interfaces de tipado TypeScript
| |-- index.ts                  # Exportador del array estructurado
| '-- pagina-web-servicios/
|     '-- index.ts              # Payload de datos de la instancia 01
|-- lib/
| |-- site-config.ts            # Variables de entorno y enrutamiento
absoluto
| |-- contact.ts                # Controlador de peticiones y validación
(Server Action)
| |-- paises.ts                 # Matriz de metadatos internacionales y
RegEx
| '-- utils.ts                  # Función cn() de fusión condicional
|-- public/
| |-- .nojekyll                 # Supresión de Jekyll para GitHub Pages
| |-- icon.svg                  # Vector de metadatos (Favicon)
| |-- og_image.png              # Asset Open Graph (1200x630 px)
| |-- robots.txt                # Directivas de rastreo (Web Crawlers)
| |-- sitemap.xml               # Mapeo topológico de rutas habilitadas
| |-- images/perfil.jpg         # Asset estático (Hero profile)
| '-- proyectos/pagina-web-servicios/
|     |-- Portada.png            # Asset principal de instancia
|     '-- Imagen-1.png           # Assets secundarios de instancia
|-- next.config.mjs              # Directivas de compilación del framework
Next.js
|-- package.json                 # Registro de módulos y directivas NPM
|-- tsconfig.json               # Parámetros de estandarización del
compilador TS
'-- postcss.config.mjs           # Motor de procesamiento CSS

```

Listing 1. Estructura del repositorio en entorno de desarrollo

Regla arquitectónica crítica: gestión de activos estáticos

Es un requerimiento estricto de la arquitectura en modo output: 'export' que los activos estáticos residan exclusivamente en el directorio `public/`. Cualquier recurso binario (imágenes) alojado en `data/` o `components/` no será procesado por el empaquetador y quedará inaccesible en los entornos de producción.

3. Configuración del Núcleo (lib/)

3.1. lib/site-config.ts — Variables globales

```
1 // Resolución dinámica del path base dependiendo del entorno de
  compilación.
2 // Producción (GitHub Pages): "/inicio" | Desarrollo: ""
3 export const basePath = process.env.NEXT_PUBLIC_BASE_PATH || ''
4
5 // Función de abstracción para la resolución de rutas estáticas de
  forma segura.
6 export const asset = (path: string) => `${basePath}${path}`
7
8 // Inyección de la variable de entorno del endpoint de recepción
  de datos.
9 export const formspreeId = process.env.NEXT_PUBLIC_FORMSPREE_ID ||
  ''
```

Listing 2. site-config.ts — Enrutamiento y control de entornos

3.2. lib/utils.ts — Fusión de clases condicionales

```
1 import { clsx, type ClassValue } from 'clsx'
2 import { twMerge } from 'tailwind-merge'
3
4 // Utilidad core para la composición dinámica de clases CSS.
5 // Resuelve la especificidad de Tailwind CSS evitando colisiones
  en tiempo de render.
6 export function cn(...inputs: ClassValue[]) {
7   return twMerge(clsx(inputs))
8 }
```

Listing 3. utils.ts — Motor de resolución CSS

3.3. lib/paises.ts — Matriz de validación internacional

El archivo instancia un controlador de más de 200 países para el enrutamiento y la validación telefónica. La arquitectura de datos es la siguiente:

Cuadro 2. Estructura de la interfaz de validación telefónica

Atributo	Tipo de dato	Propósito lógico
<code>codigo</code>	string	Estándar ISO 3166-1 alfa-2 (p. ej., "ES")
<code>nombre</code>	string	Identificador textual localizado
<code>prefijo</code>	string	Extensión de marcado internacional (p. ej., "+34")
<code>regex</code>	RegExp	Patrón de expresión regular para validación sintáctica local
<code>placeholder</code>	string	Máscara de entrada para asistencia a nivel de UI

Procedimiento de extensión del catálogo de países

La constante `TABLA` contiene las cadenas serializadas de países (formato: `ISO prefijo Nombre`). El sistema realiza un parseo estático y asigna propiedades avanzadas (`regex` y `placeholder`) en el objeto iterativo `EXTRAS`. Para modificar validaciones específicas, se debe sobrescribir la expresión regular correspondiente en dicho objeto.

3.4. `lib/contact.ts` — Controlador de peticiones del formulario

```

1 export type ContactState = {
2   status: 'idle' | 'success' | 'error'
3   message?: string
4   errors?:
      Partial<Record<'name' | 'email' | 'service' | 'subject' | 'message',
        string>>
5   values?: { name?:string; email?:string; phone?:string; /* ... */
6   }
7 }
8 export async function submitContact(
9   _prev: ContactState,
10  formData: FormData,
11 ): Promise<ContactState> {
12
13   // 1. Extracción y normalización de la carga útil (saneamiento
14     del payload)
15   const name = String(formData.get('name') ?? '').trim()
16   const email = String(formData.get('email') ?? '').trim()
17   const service = String(formData.get('service') ?? '').trim()
18   const subject = String(formData.get('_subject') ?? '').trim()
19   const message = String(formData.get('message') ?? '').trim()
20
21   // 2. Validación en el servidor (server-side validation)
22   const errors: ContactState['errors'] = {}
23   if (name.length < 2 || name.length > 100)

```

```
23     errors.name = 'Indica un nombre válido.'
24     if (!EMAIL_PATTERN.test(email) || email.length > 200)
25         errors.email = 'El formato del email es incorrecto.'
26     if (!service)
27         errors.service = 'Selección de servicio requerida.'
28     if (!subject || subject.length < 3 || subject.length > 150)
29         errors.subject = 'Longitud de asunto inválida.'
30     if (message.length < 10 || message.length > 2000)
31         errors.message = 'El mensaje no cumple los requisitos de
32             longitud mínima.'
33
34     if (Object.keys(errors).length > 0)
35         return { status: 'error', message: 'Fallo en la validación
36             sintáctica.', errors }
37
38     // 3. Verificación de inyección de entorno
39     if (!formspreeId)
40         return { status: 'error', message: 'Endpoint no definido en
41             entorno.' }
42
43     // 4. Invocación de la pasarela y control de excepciones (Fetch
44         API)
45     try {
46         const response = await
47             fetch('https://formspree.io/f/${formspreeId}', {
48                 method: 'POST',
49                 headers: { Accept: 'application/json' },
50                 body: formData,
51             })
52         if (!response.ok) throw new Error('HTTP Error Status:
53             ${response.status}')
54     } catch {
55         return { status: 'error', message: 'Error de red en el proceso
56             de envío.' }
57     }
58
59     return { status: 'success', message: 'Petición procesada
60         exitosamente, ${name}.' }
```

Listing 4. Implementación del Server Action en contact.ts

4. Análisis de Componentes de Interfaz

4.1. app/page.tsx — Ensamblado del layout principal

```
1 import { SiteHeader } from '@components/site-header'
2 import { Hero } from '@components/hero'
3 import { About } from '@components/about'
4 import { Services } from '@components/services'
5 import { Proyectos } from '@components/proyectos'
6 import { Contact } from '@components/contact'
```

```

7 import { SiteFooter } from '@components/site-footer'
8
9 export default function Page() {
10   return (
11     <>
12       <SiteHeader />
13       <main>
14         <Hero />           {/* Anclaje: id="inicio" */}
15         <About />          {/* Anclaje: id="sobre-mi" */}
16         <Services />       {/* Anclaje: id="servicios" */}
17         <Proyectos />      {/* Anclaje: id="proyectos" */}
18         <Contact />       {/* Anclaje: id="contacto" */}
19       </main>
20       <SiteFooter />
21     </>
22   )
23 }

```

Listing 5. page.tsx — Orden jerárquico de renderizado

Manipulación de estructuras de navegación

La reordenación de módulos dentro de `<main>` requiere una sincronización explícita con el componente `site-header.tsx`. El sistema de navegación local emplea anclajes de fragmentos de URL (`#inicio`, `#sobre-mi`) basados en el atributo `id` renderizado en el elemento `<section>` subyacente de cada módulo.

4.2. app/layout.tsx — Raíz del DOM y control de SEO

Define la abstracción global del HTML inicializando:

- Instanciación tipográfica a través de `next/font`, previniendo anomalías visuales de carga (FOUC).
- Metadatos estructurados (Open Graph, Twitter Card) para la integración de web scrapers y redes sociales.
- Implementación de JSON-LD (Schema.org) habilitando resultados enriquecidos de motores de búsqueda.
- Control semántico de `viewport` con colorimetría asignada mediante `themeColor`.

4.3. components/services.tsx — Matriz renderizada de servicios

Este componente expone una constante iterativa con la parametrización de los servicios ofertados. Su visualización emplea un control de renderizado condicional impulsado exclusivamente por CSS transicional (`grid-rows`).

```

1 const services = [
2   {
3     icon: Wrench,
4     label: 'Mantenimiento Informático',
5     description: 'Puesta a punto y optimización de sistemas

```

```

        operativos ' +
6      '(Windows y Linux), depuración de software, actualización de
        ' +
7      'controladores y resolución de fallos sistémicos.',
8    },
9    // ... Iteración estandarizada posterior
10 ]

```

Listing 6. Estructura de la iteración de servicios

4.4. components/proyectos.tsx — Subsistema de renderizado de proyectos

Este módulo representa el componente de mayor complejidad arquitectónica del repositorio. Orquesta cuatro subsistemas y un motor propio de transpilación de Markdown a JSX.

4.4.1. Funciones integradas

Cuadro 3. Funciones integradas en proyectos.tsx

Sub-módulo	Lógica y propósito
<code>dividirSecciones()</code>	Analizador léxico (parser) que fracciona el payload del <code>readme</code> identificando instancias de encabezados de nivel 2 ("##").
<code>renderMarkdown()</code>	Intérprete estático que transforma la notación Markdown fundamental en estructuras JSX sin requerir dependencias externas al framework base.
<code>Visor</code>	Control de estado a pantalla completa de imágenes renderizadas, implementando eventos de teclado nativos.
<code>Galería</code>	Motor de carrusel que implementa escuchas de eventos táctiles (Touch Events) y cálculo de derivadas temporales para el deslizamiento.

4.4.2. Accesibilidad en modales de proyecto

```

1 // Event listener global y bloqueo de scroll inferior del DOM
2 useEffect(() => {
3   const onKey = (e: KeyboardEvent) => { if (e.key === 'Escape')
      onClose() }
4   document.addEventListener('keydown', onKey)
5   document.body.style.overflow = 'hidden' // Bloqueo condicional
      del scroll
6   return () => {
7     document.removeEventListener('keydown', onKey)
8     document.body.style.overflow = '' // Restauración segura
      en el unmount
9   }
10 }, [onClose])

```

Listing 7. Control de interfaz de usuario en modales

5. Sistema de Procesamiento de Formularios

5.1. Validación asíncrona de estado por campo

```

1 // Tipificación estática de los tres posibles estados del
  componente de entrada
2 const fieldOk      = '...border-input...'           // Estado
  primario o no tocado
3 const fieldErr     = '...border-destructive...'     // Validación
  fallida
4 const fieldValid   = '...border-green-500...'       // Validación
  satisfactoria
5
6 // Función de abstracción de clase inyectada en el componente
7 function inputClass(touched: boolean, error: string, value:
  string) {
8   if (!touched) return fieldOk
9   if (error)     return fieldErr
10  if (value)      return fieldValid
11  return fieldOk
12 }

```

Listing 8. Asignación condicional de clases en entradas del formulario

La lógica implementa una reevaluación del estado en el evento de desenfoque del DOM (onBlur), previniendo cambios bruscos no controlados mientras el usuario edita.

5.2. Control de lectura obligatoria (bloqueo de envíos)

```

1 // El estado evalúa la profundidad de desplazamiento del nodo
  contenedor
2 const [haLeidoHastaElFinal, setHaLeidoHastaElFinal] =
  useState(false)
3 const scrollContainerRef = useRef<HTMLDivElement>(null)
4
5 const handleScrollTerminos = () => {
6   const el = scrollContainerRef.current
7   if (!el) return
8   // Evaluación de la profundidad contra el scrollHeight real
    (margen de 5px)
9   if (el.scrollTop + el.clientHeight >= el.scrollHeight - 5)
10     setHaLeidoHastaElFinal(true)
11 }

```

Listing 9. Lógica de comprobación de lectura de políticas de uso

6. Implementación de Ofuscación Anti-Spam

6.1. Generación segura de identificadores y descriptores

Para mitigar el rastreo automatizado (web scraping) de la información de contacto, la dirección electrónica del emisor no existe como texto plano en el árbol de compilación, sino que se ofusca mediante codificación Base64.

```
1 export function generarMailtoUrl(origen: string): string {
2   // Decodificación en cliente mediante la utilidad nativa 'atob'
3   const correo =
4     atob('cmVkd2FuYWxleGFuZHI1dGVvZG9yQGdtYWlsLmNvbQ==')
5
6   // Generación ligera de un identificador de contacto (UUID
7     derivado)
8   const idUnico =
9     `ID-${YYYY}${MM}${DD}-${hh}${mm}${ss}-${ms}${randomHex}-${tz}`
10 }
11
12 const asunto = `Contacto formal mediante ${origen} [${idUnico}]`
13 return `mailto:${correo}?subject=${encodeURIComponent(asunto)}`
```

Listing 10. Procedimiento de ofuscación de credenciales

Actualización del vector de contacto (Base64)

La alteración del punto de contacto requiere obligatoriamente una codificación externa en estándar Base64 (empleando `btoa('nuevo@email.com')`). La cadena resultante debe reemplazarse estrictamente en dos nodos específicos del proyecto: los componentes `mail-modal.tsx` y `contact-form.tsx`.

7. Sistema de Configuración de Datos (Proyectos)

7.1. Interfaz y entidades (data/proyectos/types.ts)

```
1 export interface ProyectoData {
2   id: string // Identificador de instancia (slug).
3   // Carpeta en public/
4   titulo: string // Cadena del nodo título de la UI
5   subtítulo: string // Cadena descriptiva de la UI
6   portada: string // Dirección estática del asset inicial
7   galeria: string[] // Array de punteros de red secundarios
8   readme: string // String multilínea con formato Markdown
9   github?: string // Puntero URL al control de versiones
10  // (opcional)
11  tecnologias: string[] // Array de tokens descriptivos de
12  // infraestructura
13 }
```

Listing 11. Tipado central del motor de proyectos

Procedimiento de integración de nuevos proyectos

1. Generación de un nuevo directorio lógico: `data/proyectos/[slug]`.
2. Creación del archivo de exportación `index.ts` cumpliendo estrictamente con la interfaz `ProyectoData`.
3. Importación modular del nuevo archivo en `data/proyectos/index.ts` e inyección en el array de exportación general.
4. Adición física de los assets estáticos en el directorio replicado `public/proyectos/[slug]`.
5. Creación de un commit de confirmación para la activación posterior del CI/CD.

8. Sistema de Renderizado y Estilos Globales

8.1. Asignación de variables nativas de CSS

Tailwind v4 utiliza internamente un motor basado en variables CSS nativas. La versión instalada aprovecha el espacio de color OKLCH para una distribución perceptual matemáticamente predecible.

```
1 :root {
2   /* Fondo general base */
3   --background: oklch(0.985 0.003 250);
4
5   /* Acento primario definido */
6   --primary: oklch(0.5 0.17 255);
7   --primary-foreground: oklch(0.99 0 0);
8
9   /* Colores de capas estructuradas de alto contraste */
10  --ink: oklch(0.22 0.025 255);
11  --ink-foreground: oklch(0.97 0.005 250);
12
13  /* Identificación de alertas sintácticas */
14  --destructive: oklch(0.577 0.245 27);
15
16  /* Geometría estructural */
17  --radius: 0.625rem;
18 }
```

Listing 12. Declaración de tokens globales en `globals.css`

9. Pipeline de Integración y Despliegue (CI/CD)

9.1. Análisis del motor YAML

```
1 name: Deploy to GitHub Pages
2
3 on:
4   push:
5     branches: [main]          # Origen del trigger
6     workflow_dispatch:        # Activación manual habilitada
7
8   permissions:
9     contents: read
10    pages: write
11    id-token: write            # Implementación OIDC
12
13 jobs:
14   build:
15     runs-on: ubuntu-latest
16     steps:
17       - uses: actions/checkout@v5          # Descarga del
18         repository
19       - uses: pnpm/action-setup@v4          # Inicialización del
20         gestor pnpm
21       - uses: actions/setup-node@v5        # Entorno V8: Node.js
22         v24
23       with:
24         node-version: 24
25         cache: pnpm
26
27       - run: pnpm install --frozen-lockfile # Consistencia
28         estricta del árbol
29
30       - id: pages
31       uses: actions/configure-pages@v5     # Lectura e inyección
32         de base_path
33
34       - run: pnpm build                    # Generación SSG
35         hacia el directorio out/
36     env:
37       NEXT_PUBLIC_BASE_PATH: ${ steps.pages.outputs.base_path }
38
39       NEXT_PUBLIC_FORMSPREE_ID: ${
40         vars.NEXT_PUBLIC_FORMSPREE_ID }
41
42       - uses: actions/upload-pages-artifact@v3 # Serialización en
43         contenedor virtual
44       with:
45         path: ./out
46
47   deploy:
48     needs: build          # Dependencia estructural y bloqueo
49     conditional
50     runs-on: ubuntu-latest
51     environment:
52       name: github-pages
53       url: ${ steps.deployment.outputs.page_url }
54     steps:
55       - id: deployment
56       uses: actions/deploy-pages@v4      # Migración de assets
57         hacia el CDN de GitHub
```

Listing 13. Directivas del archivo `deploy.yml`

Requisitos del runtime de compilación (variables de Actions)

Para posibilitar el envío de datos mediante Formspree, es obligatorio haber provisionado en la sección *Secrets and Variables* de GitHub Actions la variable `NEXT_PUBLIC_FORMSPREE_ID`. La omisión o el fallo de carga en tiempo de compilación inhabilitará el Server Action de la página de contacto.

10. Rendimiento, Optimización SEO y WCAG

10.1. Optimización de métricas centrales de rendimiento

Cuadro 4. Protocolos de optimización de infraestructura

Categoría técnica	Implementación base
SSG (Static Site Generation)	El framework Next.js precompila el árbol del DOM íntegramente en la fase de build (Time-to-Interactive optimizado).
Bloqueo FOUC	La tipografía web se inyecta globalmente mediante <code>next/font</code> , sin peticiones adicionales.
Optimización de LCP	La directiva <code><Image priority></code> impone una descarga síncrona en la capa inicial.
Animación declarativa	Las interacciones y transiciones operan sin inyección JS, aprovechando los repintados del navegador a nivel de GPU.
Markdown nativo	La función <code>renderMarkdown()</code> procesa el contenido sin incurrir en descargas de dependencias (eficiencia del bundler).

10.2. Auditoría y directivas de accesibilidad

- Inyección del atributo `aria-labelledby` apuntando a etiquetas de alto nivel de anclaje.
- Todos los elementos puramente estéticos aplican el enmascaramiento `aria-hidden="true"`.
- Las ventanas modales operan con las asignaciones `role="dialog"` y `aria-modal="true"`.
- Los contadores de eventos asíncronos y las actualizaciones de interfaz emplean `aria-live="polite"`.

Fin del documento de especificación técnica

Redwan Alexandru Teodor – Tecnico en sistemamicroinfomaticos y redes – Sevilla, España

<https://redwan-alexandru-teodor.github.io/inicio>

<https://www.linkedin.com/in/redwan-alexandru-teodor/>

Compilado en entorno tipográfico LaTeX – Revisión octubre 2026

Análisis y mapeo de los 57 componentes nativos del árbol del repositorio.